

Aufgabe 1. Zufallsberechnungen

Ein Freund möchte für die Implementierung seines neuen Indie-RPGs den Ausgang bestimmter Aktionen durch den Wurf eines 6-seitigen Würfels regeln. Er bittet dich, diesen Würfel als Python-Funktion zu implementieren.

a) Welche Möglichkeiten stehen Programmierern zu Verfügung, um ein zufälliges Ereignis zu generieren?

b) Implementiere die untenstehenden `dice_roll`-Funktionen, ohne weitere Module zu importieren.

```
import time

def dice_roll_1():
    pass

seed = 234
z = seed
def dice_roll_2():
    pass

print("dice 1")
for i in range(10):
    print(dice_roll_1())

print("dice 2")
for i in range(10):
    print(dice_roll_2())
```

c) Was sind die Vor- und Nachteile der beiden Würfelfunktionen?

Aufgabe 2: Wurzelberechnung

Wir suchen nach effektiven Berechnungen für die Quadratwurzel einer Zahl. Da irrationale Zahlen unendlich viele Nachkommastellen haben können, versuchen wir die Ergebnisse bis zu einer bestimmten Genauigkeit zu ermitteln.

a) Nutze die Python-Bibliothek `random`, um mit Zufallssuche eine Lösung zu finden.

```
import random

def g(a, x):
    pass

def sqrtRS(a, accuracy=0.001):
    pass

print(sqrtRS(2))
```

b) Implementiere die Berechnung mittels des Bisektionsalgorithmus.

```
def f(number, x):  
    pass  
  
def sqrtBS(n, a, b, accuracy=0.00001):  
    pass  
  
print(sqrtBS(2, 0, 2))
```

c) Implementiere die Berechnung mittels Gradient-Descent.

```
def g(a, x):  
    return abs(a - x**2)  
  
def sqrtGD(a, accuracy=0.00001):  
    pass  
  
print(sqrtGD(2))
```

d) Vergleiche Ergebnisse und Anzahl der Rechenschritte der drei Verfahren.

Zusatzaufgabe: Lineare Gleichungssysteme

Gegeben sind die folgenden Matrixoperatoren als Funktionen. Nutze sie, um für ein lineares Gleichungssystem $Ax = b$ eine Lösung für x zu approximieren.

Hinweis: A ist eine gegebene Matrix, b ist ein gegebener Vektor und x ist der gesuchte Vektor.

```
def matrixVectorProduct(A, x):  
    b = [0] * len(x)  
    for i in range(len(A)):  
        for j in range(len(A[0])):  
            b[i] += A[i][j] * x[j]  
    return b  
  
def vectorNormL1(x):  
    norm = 0  
    for i in range(len(x)):  
        norm += abs(x[i])  
    return norm  
  
def vectorNormL2Square(x):  
    norm = 0  
    for i in range(len(x)):  
        norm += x[i]**2  
    return norm
```

```

def vectorDifference(x, y):
    z = [0] * len(x)
    for i in range(len(x)):
        z[i] = x[i] - y[i]
    return z

def vectorProduct(a, x):
    z = [0] * len(x)
    for i in range(len(x)):
        z[i] = x[i] * a
    return z

def residuum(A, b, x):
    bTest = matrixVectorProduct(A, x)
    residuum = vectorNormL1(vectorDifference(bTest, b))
    return residuum

```

a) Löse das Problem via randomisierter Suche und teste das Ergebnis am folgenden Beispiel.

```

import random

def solveRS_Axb(A, b, accuracy=0.01):
    pass

A=[[1,2,3],
   [4,5,6],
   [7,8,9]]
b=[10,11,12]

print("Ergebnisvektor:", solveRS_Axb(A, b))

solution = [-9/2, 0, 29/6]
print("Optimale Lösung:", solution)

```

b) Nutze Gradient Descent, um die Lösung zu finden.

```

import random

def solveGD_Axb(A, b, accuracy=0.00001):
    pass

A=[[1,2,3],
   [4,5,6],
   [7,8,9]]
b=[10,11,12]

print("Ergebnisvektor:", solveGD_Axb(A, b))

```

```

solution = [-9/2, 0, 29/6]
print("Optimale Lösung:", solution)

c) Nutze Stochastic Descent, um die Lösung zu finden.

def solveSD_Axb(A, b, accuracy=0.00001):
    pass

A=[[1,2,3],
   [4,5,6],
   [7,8,9]]
b=[10,11,12]

print("Ergebnisvektor:", solveSD_Axb(A, b))

solution = [-9/2, 0, 29/6]
print("Optimale Lösung:", solution)

```